

MedIntel – AI Medical Research Assistant

Project Documentation

Version 1.0

Prepared By: Vijatesh Kadimi

Technology Stack: React • FastAPI • LangChain • Google Gemini • ChromaDB • SQLite • JWT Authentication

Deployment Platforms: Vercel (Frontend) • Render (Backend)

Table of Contents

1. Executive Summary
 2. Project Overview
 3. Problem Statement
 4. Proposed Solution
 5. Project Objectives
 6. Scope of the Project
 7. Functional Requirements
 8. Non-Functional Requirements
 9. Technology Stack
 10. System Architecture
 11. Application Workflow
 12. RAG Pipeline
 13. Authentication Module
 14. Backend Architecture
 15. Frontend Architecture
 16. API Design
 17. Deployment
 18. Testing
 19. Challenges Faced
 20. Future Enhancements
 21. Conclusion
-

1. Executive Summary

MedIntel is an AI-powered Medical Research Assistant designed to simplify the process of understanding and interacting with medical research papers. Researchers, students, healthcare professionals, and learners often spend significant time searching through lengthy PDF documents to locate specific information. Traditional keyword-based search methods frequently fail to capture the contextual meaning of user queries, making the research process time-consuming and inefficient.

To address this challenge, MedIntel leverages **Retrieval-Augmented Generation (RAG)** combined with **Google Gemini** to provide intelligent, context-aware responses based exclusively on uploaded medical documents. Instead of generating generic responses from the Large Language Model's internal knowledge, the application retrieves the most relevant sections from indexed documents and supplies them as contextual information to the model. This approach significantly improves response accuracy while reducing hallucinations.

The application offers a complete end-to-end workflow, including secure user authentication, document upload, automated PDF processing, semantic indexing using vector embeddings, conversational question answering, AI-generated document summaries, conversation history management, and cloud deployment.

The project demonstrates practical implementation of modern Generative AI technologies and showcases how Retrieval-Augmented Generation can be integrated into real-world applications to improve information retrieval and decision support.

2. Project Overview

MedIntel is a web-based AI application developed to provide an intelligent interface for interacting with medical research documents. Users can upload medical PDFs, ask natural language questions, generate concise summaries, and receive responses that are grounded in the uploaded content.

Unlike conventional chatbot applications that rely solely on pre-trained model knowledge, MedIntel incorporates a Retrieval-Augmented Generation pipeline. Each uploaded document is processed, converted into vector embeddings, stored in a vector database, and retrieved dynamically whenever the user submits a query. The retrieved content serves as contextual information for the Large Language Model, enabling highly relevant and document-specific responses.

The application follows a modular architecture, separating frontend, backend, authentication, document processing, retrieval, and AI inference into independent components. This design improves maintainability, scalability, and future extensibility.

3. Problem Statement

Medical research papers often contain extensive technical information spanning dozens or even hundreds of pages. Researchers and healthcare professionals frequently require answers to highly specific questions but must manually search through entire documents to locate relevant sections.

Traditional keyword search techniques are limited because they depend on exact word matching rather than semantic understanding. As a result, users may overlook important information even when it exists within the document.

General-purpose AI chatbots also present limitations. Since they primarily rely on pre-trained knowledge, they may generate responses that are unrelated to the uploaded document or produce inaccurate information due to hallucinations. Such behavior is particularly problematic in the medical domain, where precision and reliability are essential.

These challenges highlight the need for a system capable of understanding document content semantically and generating responses that are directly supported by the uploaded research material.

4. Proposed Solution

MedIntel addresses these challenges through a Retrieval-Augmented Generation architecture. Instead of relying solely on the Large Language Model, the application first retrieves the most relevant document sections before generating a response.

The proposed workflow includes:

- Secure user authentication using JWT.
- Upload of medical research papers in PDF format.
- Automatic extraction of textual content from uploaded documents.
- Intelligent segmentation of text into semantically meaningful chunks.
- Generation of vector embeddings for each text chunk.
- Storage of embeddings in ChromaDB.
- Semantic retrieval of relevant document sections based on user queries.
- Context-aware response generation using Google Gemini.
- Storage of conversation history for future reference.

This architecture ensures that responses remain grounded in the uploaded document rather than depending entirely on the language model's general knowledge.

5. Project Objectives

The primary objective of MedIntel is to develop an AI-powered medical research assistant capable of understanding and answering questions based on uploaded medical research papers.

Specific objectives include:

- Develop a secure authentication system for user management.
- Enable users to upload medical research documents in PDF format.
- Automatically process uploaded documents using a Retrieval-Augmented Generation pipeline.
- Generate concise AI-powered summaries of research papers.

- Allow users to interact with documents using natural language.
 - Maintain conversation history for improved user experience.
 - Deploy the application using modern cloud platforms.
 - Design a responsive and user-friendly interface suitable for both desktop and mobile devices.
-

6. Scope of the Project

The scope of MedIntel focuses on enabling intelligent interaction with uploaded medical research documents.

The application supports:

- User registration and authentication.
- Uploading medical PDF documents.
- Automatic indexing of uploaded documents.
- AI-generated summaries.
- Conversational question answering.
- Semantic document retrieval.
- Conversation history management.
- Cloud deployment.

The project intentionally restricts responses to the uploaded medical documents to ensure that generated information remains contextually relevant and minimizes hallucinations.

Current limitations include support for a single uploaded document per session and reliance on textual PDF content. Image-based medical analysis and multi-document comparison are considered future enhancements.

7. Functional Requirements

Requirement ID	Description
----------------	-------------

FR-01	User Registration
FR-02	User Login
FR-03	JWT Authentication
FR-04	Upload Medical PDF
FR-05	PDF Text Extraction
FR-06	Text Chunking

Requirement ID Description

FR-07	Embedding Generation
FR-08	ChromaDB Storage
FR-09	Semantic Search
FR-10	AI Chat
FR-11	Document Summary Generation
FR-12	Conversation History
FR-13	Logout
FR-14	Error Handling
FR-15	Responsive User Interface

8. Non-Functional Requirements

Category	Description
Performance	Fast document retrieval and response generation
Security	JWT authentication with password hashing
Scalability	Modular architecture supporting future enhancements
Reliability	Graceful error handling and recovery
Maintainability	Clean, modular codebase
Usability	Intuitive user interface with responsive design
Availability	Cloud deployment using Render and Vercel
Compatibility	Compatible with modern desktop and mobile browsers

9. Technology Stack

The MedIntel application has been developed using a modern full-stack architecture that combines web technologies, artificial intelligence, and cloud deployment services. The technology stack was selected to ensure scalability, maintainability, high performance, and seamless integration between the frontend, backend, and AI components.

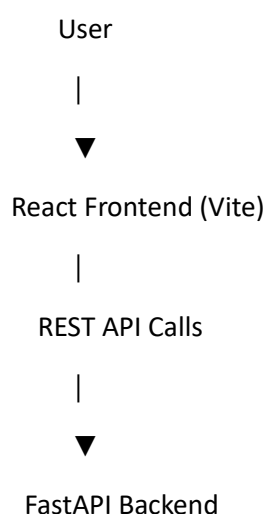
Layer	Technology	Purpose
Frontend	React.js	User Interface Development

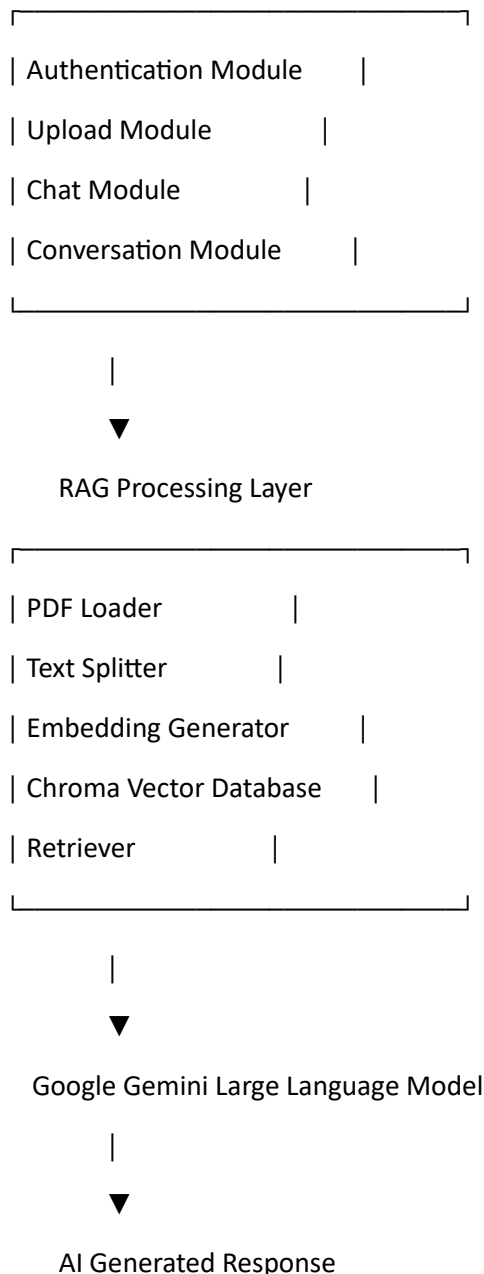
Layer	Technology	Purpose
Build Tool	Vite	Fast Development and Optimized Builds
Styling	Tailwind CSS	Responsive UI Design
Backend	FastAPI	REST API Development
Programming Language	Python	Backend Development
Authentication	JWT	Secure User Authentication
Database	SQLite	User and Conversation Storage
AI Model	Google Gemini 3.5 Flash	Context-aware Response Generation
AI Framework	LangChain	RAG Pipeline Development
Vector Database	ChromaDB	Semantic Vector Storage
PDF Processing	PyPDF	PDF Text Extraction
Deployment	Render	Backend Hosting
Deployment	Vercel	Frontend Hosting
Version Control	Git & GitHub	Source Code Management

This combination of technologies enables MedIntel to provide a secure, responsive, and intelligent platform for interacting with medical research documents.

10. System Architecture

MedIntel follows a modular client-server architecture where each component performs a dedicated responsibility. This separation improves maintainability, scalability, and simplifies future enhancements.





The architecture follows a layered approach that separates presentation, business logic, AI processing, and data storage. This modular design allows each component to evolve independently without affecting the rest of the application.

11. Application Workflow

The MedIntel application follows a structured workflow that transforms uploaded medical documents into an intelligent conversational knowledge base.

Step 1 – User Authentication

The workflow begins with secure user authentication. New users register by providing a username and password. Existing users authenticate through the login page, where credentials are verified and a JWT access token is issued for secure communication with protected backend APIs.

Step 2 – Document Upload

After successful authentication, users upload a medical research paper in PDF format. The uploaded file is securely stored within the backend before processing begins.

Step 3 – PDF Processing

The uploaded PDF is processed using PyPDFLoader, which extracts textual content while preserving the document structure. This step converts the document into machine-readable text suitable for AI processing.

Step 4 – Text Chunking

Since large language models have token limitations, the extracted text is divided into smaller semantic chunks using Recursive Character Text Splitter. Chunking improves retrieval accuracy while maintaining contextual continuity.

Step 5 – Embedding Generation

Each text chunk is converted into a high-dimensional vector representation using Google's embedding model. These embeddings capture the semantic meaning of the content rather than relying on simple keyword matching.

Step 6 – Vector Storage

Generated embeddings are stored in ChromaDB, which serves as the application's vector database. ChromaDB enables efficient similarity search during user interactions.

Step 7 – User Query

Users submit questions in natural language through the chat interface.

Step 8 – Semantic Retrieval

Instead of searching the document using keywords, the retriever identifies the most semantically relevant chunks from ChromaDB based on vector similarity.

Step 9 – AI Response Generation

The retrieved document context is combined with the user's query and forwarded to Google Gemini. The language model generates an accurate response grounded entirely in the retrieved document content.

Step 10 – Response Delivery

The generated response is displayed within the chat interface, and the conversation is stored for future reference.

12. Retrieval-Augmented Generation (RAG) Pipeline

Retrieval-Augmented Generation is the core intelligence behind MedIntel. Unlike conventional AI chatbots, the model does not rely solely on pre-trained knowledge. Instead, it retrieves relevant information from uploaded documents before generating a response.

The implemented pipeline consists of the following stages:

Medical Research PDF

|



PDF Loader

|



Text Extraction

|



Text Chunking

|



Embedding Generation

|



ChromaDB

|



Semantic Retrieval

|



Prompt Construction

|



Google Gemini LLM

|



Final AI Response

This architecture significantly improves answer accuracy while reducing hallucinations, ensuring that responses remain grounded in the uploaded document.

13. Authentication Module

Security is a critical aspect of MedIntel. The application implements JWT-based authentication to protect sensitive resources and ensure that only authenticated users can access the system.

Registration

New users create an account by providing valid credentials. Passwords are securely hashed before storage to prevent unauthorized access.

Login

During login, credentials are validated against stored user records. Upon successful authentication, the server generates a JWT access token that is returned to the client.

Authorization

Protected API endpoints require a valid JWT token in the Authorization header. Requests without valid authentication are rejected.

Logout

Users can securely terminate their session by removing the stored access token from the client.

This authentication mechanism provides secure communication between the frontend and backend while protecting confidential medical research data.

14. Backend Architecture

The backend has been developed using FastAPI and follows a modular architecture to promote code organization and maintainability.

Router Layer

The router layer exposes REST API endpoints responsible for authentication, document upload, chat interactions, health monitoring, and conversation management.

Service Layer

Business logic is encapsulated within dedicated service modules responsible for authentication, AI interaction, document processing, and conversation handling.

RAG Module

The Retrieval-Augmented Generation module performs document loading, text chunking, embedding generation, vector storage, and semantic retrieval.

Database Layer

SQLite stores user credentials and conversation history, while ChromaDB manages vector embeddings for semantic search.

This separation of concerns ensures that each component remains independent and maintainable.

15. Frontend Architecture

The frontend has been developed using React and Vite to provide a responsive and interactive user experience.

The application consists of multiple reusable components responsible for:

- User Registration
- User Login
- PDF Upload
- AI Chat Interface
- Conversation History
- Navigation
- Loading Indicators
- Error Messages

State management ensures smooth communication with backend APIs while maintaining an intuitive user experience.

The user interface follows responsive design principles, making the application accessible across desktop and mobile devices.

16. Database Design

MedIntel utilizes a hybrid data storage approach by combining a relational database (SQLite) with a vector database (ChromaDB). This architecture enables efficient management of structured application data while supporting semantic retrieval of unstructured medical documents.

SQLite is responsible for storing user information, authentication details, and conversation history. ChromaDB is dedicated to storing vector embeddings generated from uploaded medical documents, enabling semantic similarity searches during user interactions.

User Table

The User table stores registration details and authentication information for each user.

Field	Data Type	Description
id	Integer	Primary Key
username	String	Unique Username
password	String	Hashed Password

Passwords are encrypted using industry-standard hashing algorithms before storage, ensuring user credentials remain secure.

Conversation Table

The Conversation table maintains a complete record of user interactions with the chatbot.

Field	Data Type	Description
id	Integer	Primary Key
user_id	Integer	Linked User ID
question	Text	User Query
answer	Text	AI Generated Response
timestamp	DateTime	Conversation Time

Maintaining conversation history enables users to revisit previous discussions and improves the overall usability of the application.

ChromaDB Vector Store

Unlike traditional databases, ChromaDB stores high-dimensional vector embeddings rather than structured records.

Each stored document consists of:

- Document Chunk
- Vector Embedding
- Metadata
- Source Information

When a user submits a query, ChromaDB performs semantic similarity search to identify the most relevant document chunks before forwarding them to the Large Language Model.

17. API Design

The backend exposes RESTful APIs developed using FastAPI. These APIs facilitate secure communication between the frontend application and backend services.

Authentication APIs

Method	Endpoint	Description
POST	/auth/register	Register a new user
POST	/auth/login	Authenticate user and generate JWT

Document APIs

Method Endpoint Description

POST /upload Upload Medical PDF

DELETE /upload Remove uploaded document

Chat APIs

Method Endpoint Description

POST /chat Ask questions

POST /summary Generate document summary

Conversation APIs

Method Endpoint Description

GET /conversation Retrieve chat history

DELETE /conversation Delete conversation

Health API

Method Endpoint Description

GET /health Verify backend status

The API design follows REST principles, ensuring consistency, simplicity, and scalability.

18. Deployment Architecture

To ensure public accessibility and scalability, MedIntel has been deployed using modern cloud platforms.

Frontend Deployment

The React frontend is hosted on **Vercel**, providing automatic builds, global content delivery, and optimized static asset hosting.

Key advantages include:

- Continuous deployment from GitHub
- Fast content delivery
- Automatic HTTPS
- Production-ready hosting

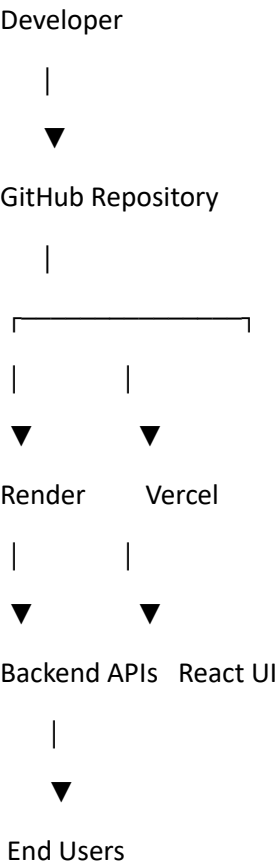
Backend Deployment

The FastAPI backend is deployed on **Render**, which hosts the REST APIs responsible for authentication, document processing, and AI interactions.

Backend deployment includes:

- Automatic GitHub integration
 - Environment variable configuration
 - Secure HTTPS endpoints
 - Automatic application restarts
-

Deployment Workflow



This deployment strategy separates frontend and backend responsibilities while maintaining efficient communication through REST APIs.

19. Testing and Validation

Extensive testing was performed to verify the correctness, reliability, and stability of the application.

Functional Testing

Test Case	Expected Result	Status
User Registration	Account Created	Passed
User Login	JWT Generated	Passed
PDF Upload	Document Indexed	Passed
Summary Generation	AI Summary Produced	Passed
AI Chat	Accurate Response	Passed
Conversation History	Data Retrieved	Passed
Logout	Session Ended	Passed

Integration Testing

The interaction between frontend, backend, database, vector store, and AI services was validated successfully.

Modules tested include:

- React ↔ FastAPI Communication
 - Authentication Flow
 - PDF Processing Pipeline
 - ChromaDB Retrieval
 - Google Gemini Integration
-

Deployment Testing

Deployment testing confirmed that:

- Frontend is accessible through Vercel.
 - Backend APIs are accessible through Render.
 - Authentication functions correctly.
 - PDF uploads operate as expected.
 - AI-generated responses are returned successfully.
-

20. Security Features

Security was considered throughout the development lifecycle.

The application implements several mechanisms to protect user data and application resources.

JWT Authentication

Each authenticated user receives a JSON Web Token that is required for accessing protected APIs.

Password Hashing

User passwords are hashed before storage, ensuring that plaintext credentials are never stored in the database.

Protected APIs

Sensitive operations such as document upload, conversation retrieval, and AI interaction require authentication.

Environment Variables

API keys, JWT secrets, and deployment configurations are securely managed using environment variables rather than hardcoded values.

Error Handling

The application gracefully handles exceptions and returns meaningful error messages without exposing sensitive system information.

21. Challenges Faced

During the development of MedIntel, several technical challenges were encountered and successfully resolved.

Deployment Issues

Deploying AI applications to cloud platforms introduced challenges related to startup time, dependency management, and environment configuration. These issues were resolved through systematic debugging and deployment optimization.

Embedding Configuration

Selecting and configuring an appropriate embedding model required experimentation to ensure compatibility with both local development and cloud deployment.

Vector Database Initialization

Proper initialization and management of ChromaDB required careful handling of document indexing and retriever creation.

Frontend–Backend Integration

Cross-Origin Resource Sharing (CORS), authentication headers, and API endpoint configuration were carefully managed to ensure seamless communication between the frontend and backend.

These challenges provided valuable experience in debugging distributed AI systems and reinforced the importance of modular design and incremental testing.

22. Future Enhancements

Although the current implementation satisfies the project objectives, several enhancements can further improve the application's capabilities.

Future improvements include:

- Support for multiple uploaded medical documents.
- Cross-document semantic search.
- Medical image analysis using multimodal AI models.
- Voice-based interaction.
- Multi-language support.
- Cloud-hosted relational databases such as PostgreSQL.
- Advanced citation generation.
- User profile management.
- Export conversation history to PDF.
- Administrative analytics dashboard.

These enhancements would increase scalability and extend the application's applicability to broader medical research scenarios.

23. Conclusion

MedIntel successfully demonstrates the practical application of Generative Artificial Intelligence within the healthcare research domain by integrating Retrieval-Augmented Generation (RAG) with a modern web-based architecture. The system enables users to upload medical research papers, generate intelligent summaries, and interact with document content through natural language while ensuring that responses remain grounded in the uploaded material.

The project combines secure authentication, semantic search, vector databases, Large Language Models, and cloud deployment into a cohesive and scalable solution. By leveraging FastAPI, React, LangChain, ChromaDB, and Google Gemini, the application delivers an efficient, reliable, and user-friendly platform for medical document exploration.

Beyond meeting the functional requirements, the project provided valuable practical experience in AI application development, backend engineering, frontend integration, cloud deployment, authentication, and Retrieval-Augmented Generation. The modular architecture adopted throughout the implementation ensures maintainability and provides a strong foundation for future enhancements.

Overall, MedIntel represents a complete end-to-end AI solution that demonstrates the effective integration of modern software engineering principles with state-of-the-art Generative AI technologies, making it a practical and scalable medical research assistant.